



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251189
Nama Lengkap	Alicia Luna Santoso
Minggu ke / Materi	15/ Fungsi Rekursif

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

Link Github : https://github.com/alc357/71251189_alicia.git

PENGERTIAN REKURSIF

Fungsi rekursif merupakan fungsi yang berisi dirinya sendiri atau fungsi yang mendefinisikan dirinya sendiri. Fungsi rekursif sering disebut dengan fungsi yang memanggil dirinya sendiri. Fungsi ini merupakan fungsi matematis yang berulang dan memiliki pola terstruktur, tetapi seringkali fungsi ini perlu diperhatikan agar dapat berhenti dan tidak menghabiskan memori. Fungsi ini harus digunakan secara hati-hati karena sifatnya yang unlimited loop sehingga dapat menyebabkan program hang up.

Fungsi rekursif akan terus berjalan sampai kondisi berhenti terpenuhi. Oleh karenanya, dalam fungsi rekursif perlu terdapat dua blok penting, yaitu blok yang menjadi titik berhenti dari sebuah proses rekursif dan blok yang memanggil dirinya sendiri. Di dalam rekursif terdapat dua bagian :

- **Base Case** : Adalah bagian yang menjadi penentu bahwa fungsi rekursif itu berhenti. Tanpa base case, fungsi akan berjalan tanpa henti dan menyebabkan stack overflow.
- **Rekursif Case** : Bagian yang berisi pemanggilan fungsi itu sendiri dengan argument yang lebih kecil atau bergerak menuju base case, sehingga proses terus berulang hingga akhirnya mencapai base case.

Fungsi rekursif akan terus berjalan sampai kondisi berhenti (base case) terpenuhi.

Ilustrasi sederhana, semisal kita menghitung faktorial dari n. Faktorial 5(5!) dapat dihitung sebagai $5 \times 4!$ dan $4!$ dihitung sebagai $4 \times 3!$ Dan seterusnya hingga mencapai $0! = 1$ yang merupakan base case. Setip langkah memanggil dirinya dengan nilai yang lebih kecil sampai kondisi yang diketahui nilainya.

KELEBIHAN DAN KEKURANGAN

Berikut merupakan kelebihan dan keunggulan fungsi rekursif :

- Kode program lebih ringkas, ringkas, dan rapi : dibandingkan dengan kode iterative dapat diselesaikan dengan kode rekursif yang jauh lebih ringkas dan mudah dibaca. Hal ini membuat kode lebih mudah dipahami dan dipelihara.
- Masalah kompleks dapat di breakdown menjadi sub masalah yang lebih kecil dalam rekursif, seperti traversal pohon (tree), algoritma divide dan conquer, dan pencarian dalam struktur data hierarkis. Rekursif mengikuti struktur masalah tersebut secara langsung.
- Modularitas yang tinggi. Setiap pemanggilan rekursif menyelesaikan sub-masalah dengan sifat independent, sehingga logika program lebih terstruktur dan modular.

Berikut kelemahan fungsi rekursif :

- Memakan lebih banyak memori. Hal ini karena setiap kali bagian dirinya dipanggil maka dibutuhkan sejumlah ruang memori tambahan untuk menyimpan frame baru di call stack, yang berisi parameter, variabel local, dan alamat kembali.
- Mengorbankan efisiensi dan kecepatan. Disebabkan oleh overhead dari pemanggilan fungsi berulang kali.
- Adanya Risiko stack overflow. Jika base case tidak tercapai atau rekursif terlalu dalam, program akan kehabisan ruang stack dan mengalami error stack overflow.
- Sulit untuk melakukan debugging dan kadang sulit untuk dipahami. Alur eksekusi rekursif lebih sulit dilacak dibanding iterative, terutama rekursif berlapis-lapis.

BENTUK UMUM DAN STUDI KASUS

Berikut merupakan bentuk umum dari rekursif dalam python :

```
def nama_fungsi (parameter_list) :  
    ...  
    nama_fungsi (...)
```

Gambar 1.1 : Bentuk umum rekursif

Sebetulnya semua fungsi rekursif pasti memiliki Solusi iteratifnya. Contohnya pada kasus faktorial berikut : Faktorial berfungsi untuk menghitung perkalian deret angka $1 \times 2 \times 3 \times \dots \times n$. Algoritma untuk menghitung faktorial adalah :

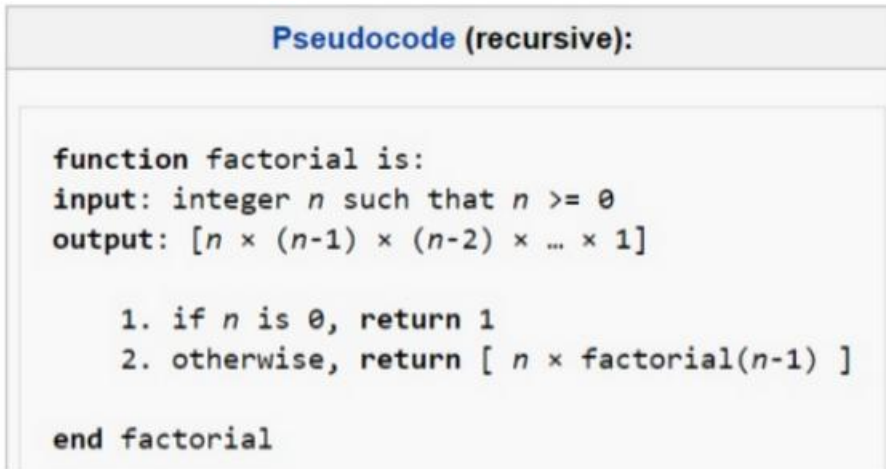
1. Menanyakan n
2. Menyiapkan variabel total untuk menampung hasil perkalian faktorial dan set nilai awal dengan 0
3. Lakukan looping dari $i = 1$ hingga n untuk mengerjakan :
4. $total = total * i$
5. Menampilkan nilai dari variabel total

Dengan menggunakan fungsi rekursif maka faktorial dapat dihitung dengan rumus yaitu :

$$\text{fact}(n) = \begin{cases} 1 & \text{if } n = 0 \\ n \cdot \text{fact}(n - 1) & \text{if } n > 0 \end{cases}$$

Gambar 1.2 : Rumus Faktorial

Sesuai dengan rumus itu dapat dibuat pseudocode secara rekursif



Gambar 1.3 : Pseudocode rekursif faktorial

Dari pseudocode itu, dapat dibuat kode python seperti berikut :

```
def faktorial(n) :
    if n==0 or n==1 :
        return 1
    else :
        return faktorial (n-1) * n

print (faktorial(4))
```

Gambar 1.4 : Kode dari perhitungan faktorial

Hasil akhirnya adalah 24

Bagaimana proses perhitungannya :

Berikut adalah gambarannya :

```
1.
2. calc_factorial(4)           # 1st call with 4
3. 4 * calc_factorial(3)       # 2nd call with 3
4. 4 * 3 * calc_factorial(2)   # 3rd call with 2
5. 4 * 3 * 2 * calc_factorial(1) # 4th call with 1
6. 4 * 3 * 2 * 1               # return from 4th call as number=1
7. 4 * 3 * 2                   # return from 3rd call
8. 4 * 6                       # return from 2nd call
9. 24                          # return from 1st call
```

Gambar 1.5 : Proses perhitungan faktorial rekursif

PERBANDINGAN REKURSIF DAN ITERATIF

Setiap masalah yang dapat diselesaikan secara rekursif juga dapat diselesaikan secara iterative (menggunakan perulangan), dan sebaliknya. Namun keduanya memiliki karakter yang berbeda. Berikut adalah perbandingan antara pendekatan rekursif dan iterative menggunakan kasus faktorial :

Pendekatan Iteratif :

```
def faktorial_iteratif(n) :  
    total = 1  
    for i in range (1, n+1) :  
        total = total * i  
    return total  
  
print (faktorial_iteratif(4))
```

Gambar 1.6 : Gambar penyelesaian soal faktorial dengan pendekatan iteratif

Pendekatan Rekursif :

```
def faktorial_iteratif(n) :  
    if n == 0 or n == 1 :  
        return 1  
    else :  
        return faktorial_iteratif(n-1) * n  
  
print(faktorial_iteratif(4))
```

Gambar 1.7 : Gambar penyelesaian soal faktorial dengan pendekatan rekursif

ASPEK	REKURSIF	ITERATIF
Keterbacaan kode	Singkat dan elegan	Panjang namun eksplisit
Penggunaan Memori	Lebih besar (call stack)	Hemat
Kecepatan eksekusi	Lebih lambat (overhead)	Cepat
Risiko error	Risiko Stack overflow	Tidak berisiko stack overflow
Cocok untuk	Masalah hirarki/pohon	Masalah linear sederhana
Debugging	Lebih sulit dilacak	Lebih mudah dilacak

JENIS-JENIS REKURSIF

Berikut merupakan jenis-jenis rekursif yang perlu dipahami :

1. Direct Recursion (rekursif Langsung) : Terjadi ketika sebuah fungsi memanggil dirinya sendiri secara langsung. Contoh penggunaannya adalah pada fungsi fibonacci dan faktorial
2. Indirect Recursion (rekursi tidak langsung) : Terjadi ketika fungsi A memanggil fungsi B dan fungsi B memanggil kembali fungsi A. Pemanggilan terjadi tidak secara langsung namun melalui perantara.
3. Tail Recursion (rekursi ekor) : jenis rekursif dimana pemanggilan rekursif merupakan opsi terakhir yang dilakukan oleh fungsi sebelum mengembalikan nilai. Tidak ada komputasi yang dilakukan setelah pemanggilan rekursif tersebut.

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

Link Github : https://github.com/alc357/71251189_alicia.git

SOAL 1

Source Code :

```
def prima (n, i=2) :  
    if n < 2 :  
        return False  
    if i * i > n :  
        return True  
    if n % i == 0 :  
        return False  
    return prima(n,i+1)  
  
n = int(input("Masukan bilangan : "))  
if prima(n) :  
    print(f"{n} adalah bilangan prima")  
else :  
    print(f"{n} bukan bilangan prima")
```

Langkah – langkah :

1. Pertama, kita definisikan fungsi rekursif dengan nama prima dengan dua parameter yaitu n sebagai bilangan yang ingin dicek dan i sebagai pembagi yang dimulai dari nilai default
- 2.

2. Kedua, kita buat kondisi pertama dengan if $n < 2$ sebagai basis rekursi pertama. Jika bilangan n kurang dari 2, maka fungsi langsung mengembalikan nilai false karena bilangan 0 dan 1 bukan bilangan prima.
3. Ketiga, kita buat kondisi kedua dengan if $i * i < n$ sebagai basis rekursi kedua. Jika nilai i dikali i sudah melebihi n , artinya tidak ditemukan pembagi, maka fungsi mengembalikan nilai True karena bilangan tersebut adalah prima
4. Keempat, kita buat kondisi ketiga dengan if $n \% i == 0$ sebagai basis rekursi ketiga. Jika n habis dibagi dengan i (sisa bagi sama dengan 0) maka ditemukan pembagi artinya fungsi mengembalikan nilai false karena bilangan tersebut bukan bilangan prima.
5. Kelima, jika tidak ada satupun kondisi basis yang terpenuhi, fungsi memanggil dirinya sendiri secara rekursif dengan $\text{prima}(n, i+1)$ yaitu mengecek pembagi berikutnya dengan nilai i ditambah dengan 1
6. Keenam, di luar fungsi kita meminta input dari user menggunakan input () yang kita konversi menjadi integer menggunakan int() dan kita simpan dalam variabel n .
7. Ketujuh, kita panggil fungsi $\text{prima}(n)$ dalam kondisi if. Jika fungsi mengembalikan True, program mencetak pesan bahwa n adalah bilangan prima menggunakan f-string, jika tidak maka akan mencetak bahwa n bukan bilangan prima.

Output :

```
Masukan bilangan : 5
5 adalah bilangan prima
PS C:\prak_alpro\71251189_alic
> & "C:/Users/alc 357/AppData/
.exe" c:/prak_alpro/71251189_e
u15/soal01.py
Masukan bilangan : 1
1 bukan bilangan prima
PS C:\prak_alpro\71251189_alic
> & "C:/Users/alc 357/AppData/
.exe" c:/prak_alpro/71251189_e
u15/soal01.py
Masukan bilangan : 2
2 adalah bilangan prima
..
```

Gambar 2.1 : Output program soal nomor 1

```

def prima (n, i=2) :
    if n < 2 :
        return False
    if i * i > n :
        return True
    if n % i == 0 :
        return False
    return prima(n,i+1)

n = int(input("Masukan bilangan : "))
if prima(n) :
    print(f"{n} adalah bilangan prima")
else :
    print(f"{n} bukan bilangan prima")

```

Gambar 2.2 : Kode Program soal nomor 1

SOAL 2

Source Code :

```

def palindrom(kalimat) :

    kalimat = kalimat.replace(" ", "").lower()

    return cek(kalimat)

def cek (s) :

    if len(s) <= 1 :

        return True

    if s [0] != s[-1] :

        return False

    return cek(s[1:-1])

kalimat = input("Masukkan kalimat : ")

if palindrom(kalimat) :

    print(f"{kalimat} adalah palindrom")

```

```
else :
```

```
    print(f"{kalimat} bukan palindrom")
```

Langkah – langkah :

1. Pertama, kita definisikan fungsi palindrom dengan satu parameter yaitu kalimat sebagai fungsi pembersih. Dalam fungsi itu kita menghapus semua spasi menggunakan `.replace()` dan mengubah seluruh huruf menjadi huruf kecil menggunakan `.lower()` dan hasilnya kita simpan kembali ke variabel kalimat
2. Kedua, fungsi palidrom kemudian memanggil fungsi rekursif utama bernama cek dengan meneruskan kalimat yang sudah dibersihkan tadi sebagai argumen dan langsung mengembalikan hasilnya.
3. Ketiga, kita definisikan fungsi rekursif cek dengan parameter s yang menerima string yang akan diperiksa.
4. Keempat, kita buat kondisi pertama `if len(s) <= 1` sebagai basis rekursi pertama. Jika panjang string sudah 0 atau 1 karakter maka fungsi langsung mengembalikan True karena string kosong atau satu huruf sudah pasti palindrom
5. Kelima, kita buat kondisi kedua `if s[0] != s[-1]` sebagai basis rekursi kedua. Jika huruf pertama tidak sama dengan huruf terakhir maka fungsi langsung mengembalikan False karena kalimat tersebut sudah pasti bukan palindrom.
6. Keenam, jika kedua kondisi basis tidak terpenuhi, fungsi memanggil dirinya sendiri secara rekursif dengan cek `(s[1:-1])`, yaitu memotong satu huruf dari depan dan satu huruf dari belakang, lalu mengecek sisa bagian tengahnya.
7. Ketujuh, di luar fungsi kita minta input kalimat dari user menggunakan fungsi `input()` dan kita simpan dalam variabel kalimat.
8. Kedelapan, kita panggil fungsi `palindrom(kalimat)` dalam kondisi `if`. Jika fungsi mengembalikan True, program mencetak pesan bahwa kalimat tersebut adalah palindrom menggunakan f-string. Jika tidak, blok `else` akan mencetak pesan bahwa kalimat tersebut bukan palindrom

Output :

```
~/soal02.py
Masukkan kalimat : kasurruusak
kasurruusak adalah palindrom
PS C:\prak_alpro\71251189_alicia\Pr
> & "C:/Users/alc 357/AppData/Local
.exe" c:/prak_alpro/71251189_alicia
u15/soal02.py
Masukkan kalimat : ayam goreng
ayam goreng bukan palindrom
PS C:\prak_alpro\71251189_alicia\Pr
> & "C:/Users/alc 357/AppData/Local
.exe" c:/prak_alpro/71251189_alicia
u15/soal02.py
Masukkan kalimat : ussu
ussu adalah palindrom
```

Gambar 2.3 : Output dari soal nomor 2

```
def palindrom(kalimat) :
    kalimat = kalimat.replace(" ", "").lower()
    return cek(kalimat)

def cek (s) :
    if len(s) <= 1 :
        return True
    if s [0] != s[-1] :
        return False
    return cek(s[1:-1])

kalimat = input("Masukkan kalimat : ")
if palindrom(kalimat) :
    print(f"{kalimat} adalah palindrom")
else :
    print(f"{kalimat} bukan palindrom")
```

Gambar 2.4 : Kode soal nomor 2

SOAL 3

Source Code :

```

def ganjil (n) :

    if n == 1 :

        return 1

    return (2*n-1) + ganjil(n-1)

n = int(input("Masukan jumlah suku (n) : "))

hasil = ganjil(n)

deret = [str(2 * i -1) for i in range (1,n+1)]

print("deret :", "+".join(deret))

print (f"Jumlah : {hasil}")

```

Langkah – langkah :

1. Pertama, kita definisikan fungsi rekursif bernama ganjil dengan satu parameter n yang menyatakan jumlah suku deret yang ingin dijumlahkan
2. Kedua, kita buat kondisi if n == 1 sebagai basis rekursi. Jika n bernilai 1 maka fungsi langsung mengembalikan nilai 1 karena suku pertama dari deret ganjil adalah 1.
3. Ketiga, jika n lebih dari 1 fungsi mengembalikan nilai $(2*n-1) + \text{ganjil}(n-1)$, yaitu menjumlahkan suku ke-n menggunakan rumus $2*n-1$ dengan hasil pemanggilan rekursif ganjil(n-1) untuk menghitung jumlah suku-suku sebelumnya.
4. Keempat, di luar fungsi kita meminta input dari user menggunakan fungsi input() yang kita konversi menjadi integer menggunakan fungsi int(), dan kita simpan dalam variabel n.
5. Kelima, kita panggil fungsi ganjil(n) dan menyimpan hasilnya ke dalam variabel hasil.
6. Keenam, kita membuat list deret menggunakan list comprehension dengan rumus $2 * i - 1$ untuk setiap i dalam rentang 1 sampai n lalu setiap elemen kita konversi menjadi string menggunakan str().

7. Ketujuh, kita mencetak tampilan deret dengan menggunakan `"+"`.join(deret) untuk menggabungkan semua elemen list deret menjadi satu string yang dipisahkan tanda + lalu mencetaknya dengan label "deret:"
8. Kedelapan, program mencetak jumlah total deret menggunakan fungsi print dengan f string yang menampilkan nilai dari variabel hasil dengan label "Jumlah :"

Output :

```
Masukan jumlah suku (n) : 5
deret : 1+3+5+7+9
Jumlah : 25
PS C:\prak_alpro\71251189_alicia\Pr
> & "C:/Users/alc 357/AppData/Local,
.exe" c:/prak_alpro/71251189_alicia,
u15/soal03.py
Masukan jumlah suku (n) : 10
deret : 1+3+5+7+9+11+13+15+17+19
Jumlah : 100
```

Gambar 2.5 : Output soal no 3

```
def ganjil (n) :
    if n == 1 :
        return 1
    return (2*n-1) + ganjil(n-1)

n = int(input("Masukan jumlah suku (n) : "))
hasil = ganjil(n)

deret = [str(2 * i -1) for i in range (1,n+1)]
print("deret :", "+".join(deret))
print (f"Jumlah : {hasil}")
```

Gambar 2.6 : Kode Program soal nomor 3

SOAL 4

Source Code :

```
def digit(bilangan) :

    bilangan = str(bilangan)
```

```

    if len(bilangan) == 1 :

        return int(bilangan)

    return int(bilangan[0]) + digit(bilangan[1:])

bilangan = input("Masukkan bilangan : ")

hasil = digit(bilangan)

tampil = "+".join(list(bilangan))

print(f"Penjumlahan : {tampil} = {hasil}")

```

Langkah – langkah :

1. Pertama, kita definisikan fungsi rekursif bernama digit dengan satu parameter bilangan yang menyatakan bilangan yang ingin dihitung jumlah digitnya.
2. Kedua, dalam fungsi kita konversi parameter bilangan menjadi string dan menyimpan kembali dalam variabel bilangan agar setiap digit bisa diakses satu persatu menggunakan indeks.
3. Ketiga kita buat kondisi jika jumlah bilangan sama dengan 1 sebagai basis rekursi. Jika panjang string sudah tersisa 1 karakter, maka fungsi langsung mengembalikan nilai digit tersebut setelah dikonversi kembali menjadi integer.
4. Keempat, jika string masih memiliki lebih dari 1 karakter, fungsi mengembalikan nilai `int(bilangan[0]) + digit(bilangan[1:])` yang memproses sisa digit berikutnya
5. Kelima, di luar fungsi kita meminta input dari user menggunakan fungsi `input()` dan menyimpannya dalam variabel bilangan tanpa konversi `int()` agar format string tetap terjaga untuk keperluan tampilan.
6. Keenam, kita memanggil fungsi `digit(bilangan)` dan menyimpan hasilnya ke dalam variabel `hasil`.

7. Ketujuh, kita membuat variabel tampil dengan memisahkan setiap karakter dalam bilangan menggunakan list(bilangan) lalu menggabungkannya dengan "+".join(...) untuk menampilkan proses penjumlahan digit secara lengkap
8. Kedelapan, program mencetak hasil akhir menggunakan fungsi print dengan f-string yang menampilkan penjumlahan digit beserta total hasilnya.

Output :

```
Masukkan bilangan : 234
Penjumlahan : 2+3+4 = 9
PS C:\prak_alpro\71251189_alic
> & "C:/Users/alc 357/AppData/
.exe" c:/prak_alpro/71251189_a
u15/soal04.py
Masukkan bilangan : 357
Penjumlahan : 3+5+7 = 15
```

Gambar 2.7 : Output soal no 4

```
def digit(bilangan) :
    bilangan = str(bilangan)
    if len(bilangan) == 1 :
        return int(bilangan)
    return int(bilangan[0]) + digit(bilangan[1:])

bilangan = input("Masukkan bilangan : ")
hasil = digit(bilangan)

tampil = "+".join(list(bilangan))
print(f"Penjumlahan : {tampil} = {hasil}")
```

Gambar 2.8 : Kode Program soal nomor 4

SOAL 5

Source Code :

```
def faktorial (n) :

    if n == 0 or n == 1 :

        return 1
```



```

    return n * faktorial (n-1)

def kombinasi (n,r) :

    if r==0 or r == n :

        return 1

    return kombinasi(n-1, r-1) + kombinasi(n-1,r)

n = int(input("Masukkan nilai n : "))

r = int(input("Masukkan nilai r : "))

if r > n :

    print("r tidak boleh lebih besar dari n")

else :

    hasil = kombinasi(n,r)

    print(f"C({n}, {r}) = {n}! / {r}! x ({n}-{r})!")

    print(f"C({n}, {r}) = {hasil}")

```

Langkah – langkah :

1. Pertama, kita definisikan fungsi rekursif pertama bernama faktorial dengan parameter n. Jika n bernilai 0 atau 1 maka fungsi mengembalikan nilai 1 sebagai basis rekursi, jika tidak fungsi mengembalikan $n * \text{faktorial}(n-1)$ secara rekursif
2. Kedua, kita definisikan fungsi rekursif utama bernama kombinasi dengan dua parameter yaitu n sebagai jumlah total elemen dan r sebagai jumlah elemen yang dipilih
3. Ketiga kita buat kondisi $\text{if } r == 0 \text{ or } r == n$ sebagai basis rekursi fungsi kombinasi. Jika r bernilai 0 atau r sama dengan n, maka fungsi langsung mengembalikan nilai 1 karena $C(n,0)$ dan $C(n,n)$ selalu bernilai 1.

4. Keempat, jika kondisi basis tidak terpenuhi, fungsi mengembalikan kombinasi $(n-1, r-1)$ + kombinasi $(n-1, r)$ yang merupakan rumus segitiga pascal. Rumus ini menyatakan bahwa $C(n, r)$ dapat dihitung dari penjumlahan dua kombinasi yang lebih kecil secara rekursif
5. Kelima, di luar fungsi kita meminta dua input dari user menggunakan fungsi `input()` yang masing-masing dikonversi menjadi integer menggunakan `int()` lalu disimpan dalam variabel `n` dan `r`
6. Keenam, kita buat kondisi `if r > n` untuk memvalidasi input. Jika `r` lebih besar dari `n` maka program mencetak karena kombinasi tidak dapat dihitung.
7. Ketujuh, jika input valid, kita memanggil fungsi kombinasi (n, r) dan menyimpan hasilnya ke dalam variabel `hasil`
8. Kedelapan, program mencetak dua baris output menggunakan f-string. Baris pertama menampilkan rumus lengkap dan baris kedua menampilkan nilai hasil akhir kombinasi.

Output

```
Masukkan nilai n : 4
Masukkan nilai r : 5
r tidak boleh lebih besar dari n
PS C:\prak_alpro\71251189_alicia> & "C:/Users/alc 357/AppData/Local/Programs/Python/Python39-64/Python.exe" c:/prak_alpro/71251189_alicia/soal05.py
Masukkan nilai n : 5
Masukkan nilai r : 2
C(5, 2) = 5! / 2! x (5-2)!)
C(5, 2) = 10
```

Gambar 2.9 : Output soal no 5

```

▼ def faktorial (n) :
▼     if n == 0 or n == 1 :
        return 1
    return n * faktorial (n-1)

▼ def kombinasi (n,r) :
▼     if r==0 or r == n :
        return 1
    return kombinasi(n-1, r-1) + kombinasi(n-1,r)

n = int(input("Masukkan nilai n : "))
r = int(input("Masukkan nilai r : "))

▼ if r > n :
    print("r tidak boleh lebih besar dari n")
▼ else :
    hasil = kombinasi(n,r)
    print(f"C({n}, {r}) = {n}! / {r}! x ({n}-{r})!")
    print(f"C({n}, {r}) = {hasil}")

```

Gambar 2.10 : Kode Program soal nomor 5